Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....5
1.1 Описание входных данных.....6
1.2 Описание выходных данных.....6
2 МЕТОД РЕШЕНИЯ.....7
3 ОПИСАНИЕ АЛГОРИТМОВ.....9
3.1 Алгоритм конструктора класса Character.....9
3.2 Алгоритм метода heal класса Healer.....9
3.3 Алгоритм функции printCharacter.....10
3.4 Алгоритм функции main.....10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....12
5 КОД ПРОГРАММЫ.....14
5.1 Файл Character.cpp.....14
5.2 Файл Character.h.....14
5.3 Файл Healer.cpp.....15
5.4 Файл Healer.h.....15
5.5 Файл main.cpp.....15
6 ТЕСТИРОВАНИЕ.....17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....18

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программную систему на языке C++, моделирующую объект персонажа игры, обладающего именем и внутренними характеристиками. Доступ к данным объекта должен осуществляться с использованием механизма friend.

Требования к классу Персонажа

Класс Персонаж должен содержать:

- три свойства:
  - имя героя (тип строковый);
  - уровень здоровья (целочисленный тип);
  - уровень маны (целочисленный тип);
- все свойства должны иметь **закрытый модификатор доступа**;
- конструктор, инициализирующий значения всех свойств.

Использование публичных методов для получения или изменения значений свойств (get, set) **не допускается**.

Реализовать функцию вывода исходного состояния объекта, которая:

- принимает объект класса Персонаж по константной ссылке;
- выводит на экран имя героя, значение здоровья и значение маны;
- имеет доступ к закрытым членам класса Персонаж.

Реализовать класс Лекарь, который:

- содержит метод, изменяющий состояние объекта класса Персонаж:
  - увеличивает значение здоровья на 20;
  - увеличивает значение маны на 10;
- имеет доступ к закрытым членам класса Персонаж.

Алгоритм работы программы

1. Ввести с клавиатуры имя героя, начальное значение здоровья и начальное значение маны.

2. Создать объект класса Персонаж, используя введённые значения.

3. Вызвать функцию для вывода исходного состояния объекта.

4. Создать объект класса Лекарь.

5. Передать объект класса Персонаж в метод класса Лекарь для изменения его состояния.

6. Повторно вызвать функцию для вывода исходного состояния объекта.

Ограничения

- Прямой доступ к закрытым свойствам класса Персонаж вне дружественных сущностей запрещён.
- Использование публичных геттеров и сеттеров запрещено.
- Обращение к закрытым членам класса Персонаж из функции main запрещено.
- Использование механизма friend допускается только в рамках требований задания.

1.1 Описание входных данных

Значения через пробел: <Имя> <здоровье> <мана>

1.2 Описание выходных данных

Первая строка: "Hero: <Имя> | Health = <значение здоровья> | Mana = <значение маны>"

Вторая строка: "Hero: <Имя> | Health = <значение здоровья> | Mana = <значение маны>"

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект hero класса Character предназначен для объект персонажа, созданный в функции main на основе введённых данных. Используется для хранения состояния и передачи в функции вывода и лечения;
- объект healer класса Healer предназначен для объект лекаря, созданный в функции main. Предназначен для вызова heal, изменяющего состояние персонажа;
- функция printCharacter для глобальная функция, дружественная классу Character. Предназначена для вывода на экран текущего состояния персонажа.

Класс Character:

- свойства/поля:
  - поле поле имя героя:
    - наименование — name;
    - тип — строка;
    - модификатор доступа — private;
  - поле поле здоровье:
    - наименование — health;
    - тип — целочисленный;
    - модификатор доступа — private;
  - поле поле мана:
    - наименование — mana;
    - тип — целочисленный;
    - модификатор доступа — private;
- функционал:

- метод Character — инициализирует объект персонажа переданными значениями имени, здоровья и маны.

Класс Healer:

- функционал:
  - метод heal — метод, принимающий ссылку на объект Character. Увеличивает здоровье персонажа на 20, а ману на 10. Имеет прямой доступ к закрытым полям благодаря дружественности класса Healer к Character.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Character			Класс, представляющий игрового персонажа	
2	Healer			Класс лекаря, который может изменять состояние персонажа	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Character

Функционал: инициализирует объект персонажа переданными значениями имени, здоровья и маны.

Параметры: строка n - имя героя, целочисленное h - уровень здоровья героя, целочисленное m - уровень маны героя.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса Character

№	Предикат	Действия	№ перехода
1		Присвоить закрытому полю name класса Character значение параметра n	2
2		Присвоить закрытому полю health класса Character значение параметра h	3
3		Присвоить закрытому полю mana класса Character значение параметра m	∅

3.2 Алгоритм метода heal класса Healer

Функционал: метод, принимающий ссылку на объект Character. Увеличивает здоровье персонажа на 20, а ману на 10. Имеет прямой доступ к закрытым полям благодаря дружественности класса Healer к Character.

Параметры: Character& c - ссылка на объект персонажа, состояние которого

будет изменено.

Возвращаемое значение: нет.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода heal класса Healer

№	Предикат	Действия	№ перехода
1		Увеличить закрытое поле health объекта c на 20: c.health += 20	2
2		Увеличить закрытое поле mana объекта c на 10: c.mana += 10	∅

3.3 Алгоритм функции printCharacter

Функционал: Вывод на экран строки с именем, здоровьем и маной персонажа. Обращение к закрытым полям объекта c осуществляется благодаря дружественности функции к классу Character.

Параметры: const Character& s - константная ссылка на объект персонажа.

Возвращаемое значение: нет.

Алгоритм функции представлен в таблице 4.

Таблица 4 – Алгоритм функции printCharacter

№	Предикат	Действия	№ перехода
1		Вывод строки, содержащей значение закрытого поля c.name, затем " | Health = ", значение закрытого поля c.health, затем " | Mana = ", значение закрытого поля c.mana	∅

3.4 Алгоритм функции main

Функционал: Основная функция программы.

Параметры: нет.

Возвращаемое значение: Целочисленное значение, код завершения программы.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		Объяление переменных: строка name, целочисленные health и mana	2
2		Ввод значений для переменных name, health, mana	3
3		Создать объект hero класса Character, передав конструктору значения name, health, mana	4
4		Вызвать функцию printCharacter(hero) для вывода исходного состояния	5
5		Создать объект healer класса Healer	6
6		Вызвать метод healer.heal(hero) для изменения состояния персонажа	7
7		Вызвать функцию printCharacter(hero) для вывода состояния после лечения	8
8		Возврат 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-2.

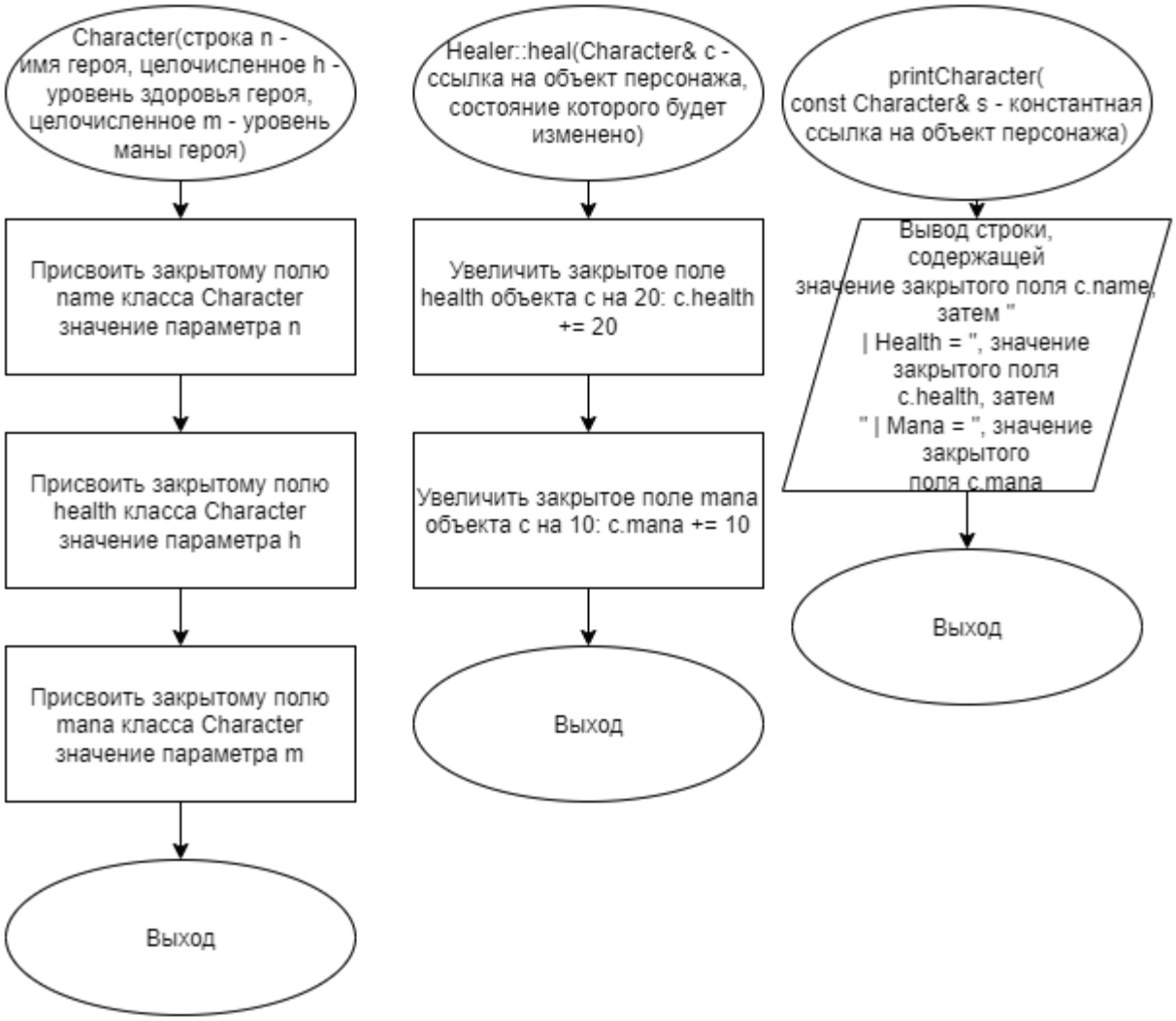

Рисунок 1 – Блок-схема алгоритма

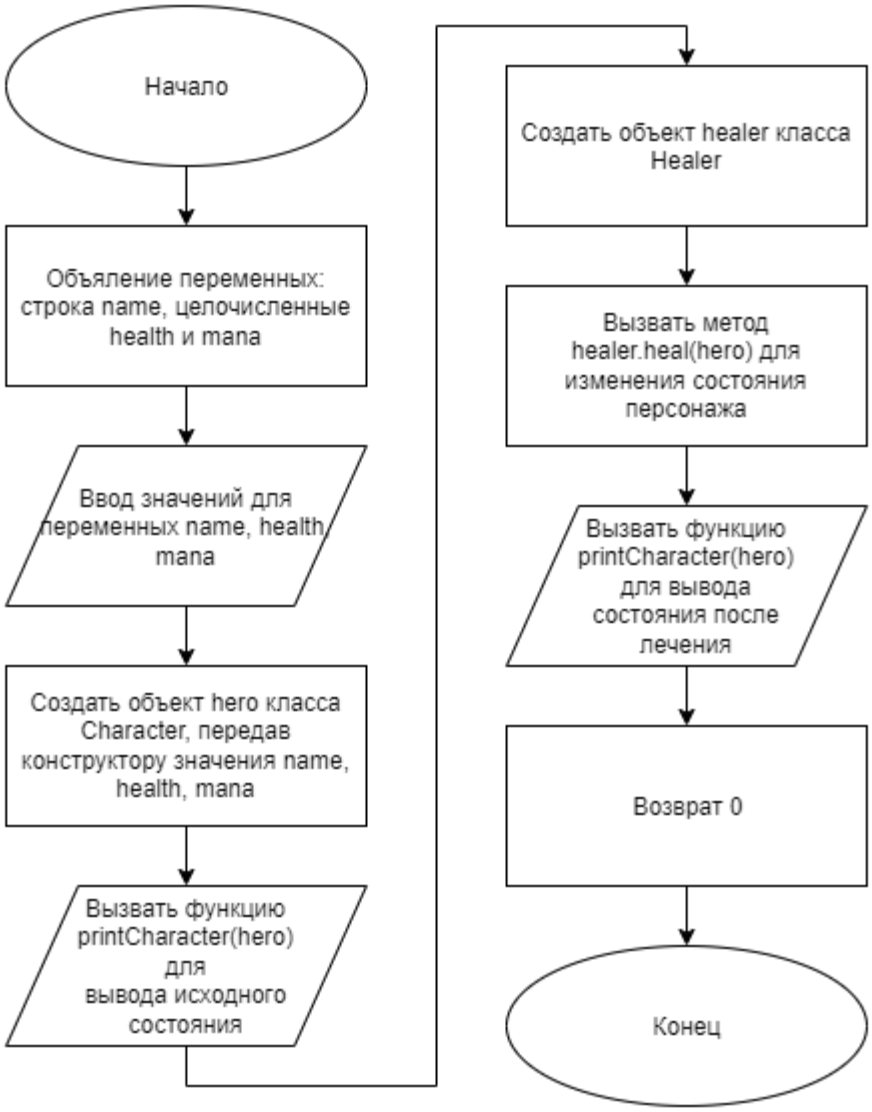

Рисунок 2 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Character.cpp

Листинг 1 – Character.cpp

```
#include "Character.h"
#include <iostream>

Character::Character(const std::string& n, int h, int m)
   :name(n), health(h), mana(m){}

void printCharacter(const Character& c){
   std::cout << "Hero: " << c.name << " | Health = " << c.health << " | Mana
= " << c.mana << std::endl;
}
```

5.2 Файл Character.h

Листинг 2 – Character.h

```
#ifndef __CHARACTER__H
#define __CHARACTER__H

#include <string>

class Healer;

class Character {
   private:
      std::string name;
      int health;
      int mana;

      friend void printCharacter(const Character& c);
      friend class Healer;

   public:
      Character(const std::string& n, int h, int m);
```

```
};

void printCharacter(const Character& c);

#endif
```

5.3 Файл Healer.cpp

Листинг 3 – Healer.cpp

```
#include "Healer.h"

void Healer::heal(Character& c){
   c.health += 20;
   c.mana += 10;
}
```

5.4 Файл Healer.h

Листинг 4 – Healer.h

```
#ifndef __HEALTH__H
#define __HEALTH__H

#include "Character.h"

class Healer{
   public:
      void heal(Character& c);
};

#endif
```

5.5 Файл main.cpp

Листинг 5 – main.cpp

```
#include <iostream>
#include <string>
```

```
#include "Character.h"
#include "Healer.h"

using namespace std;

int main()
{
   string name;
   int health, mana;

   cin >> name >> health >> mana;

   Character hero(name, health, mana);

   printCharacter(hero);

   Healer healer;
   healer.heal(hero);

   printCharacter(hero);
   return(0);
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 6.

Таблица 6 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Potter 100 50	Hero: Potter | Health = 100 | Mana = 50 Hero: Potter | Health = 120 | Mana = 60	Hero: Potter | Health = 100 | Mana = 50 Hero: Potter | Health = 120 | Mana = 60

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.

2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avrora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).

3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avrora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).

4. Шилдт Г. C++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.

5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».

6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).